LEADERS UNIVERSITY / LICENCE BI

Construire une BI avec Codex

Atlas Papeterie · 15 h · cinq séances pour construire, vérifier et expliquer

15 h	70 %	0
5 SÉANCES DE 3 H	PRATIQUE HORS PAUSES	PRÉREQUIS IT

ATLAS PAPETERIE

Codex + CSV et SQL → Entrepôt → Power BI

02 / CONSTRUIRE SA BI AVEC IA

Youssef Douma

Édition pédagogique · 27 octobre 2026 · données fictives

DÉPART / UN PROJET QUI GRANDIT

Une première mission chez Atlas

Vous rejoignez l'équipe BI d'Atlas Papeterie. Cette entreprise fictive vend des carnets, stylos, classeurs et surligneurs à Nabeul et Tunis. Vous débutez : les fichiers, le code et les mots nouveaux seront expliqués au moment utile.

Le message de la direction

« Nos ventes arrivent dans un fichier et dans une base SQL. Certaines lignes sont fausses ou répétées. Le 8 janvier, Nabeul change de zone commerciale. Préparez un rapport Power BI qui garde les bons chiffres et la bonne zone pour chaque vente. Montrez comment vous l'avez vérifié. »

Toute la chaîne, un même projet

Question : quelles ventes sont fiables, et dans quelle zone commerciale ?



Une vente = une ligne de fact_sales. Les montants restent en millimes entiers.

Deux sources, une préparation contrôlée, quatre tables puis un vrai modèle Power BI Windows.

Cette demande paraît grande. Nous la découpons en petites tâches : prévoir un résultat, demander un plan à Codex, construire une étape, exécuter puis expliquer une preuve. Vous recevez une ossature : vous n'avez pas à inventer un système complet.

La réussite visible

À la fin : 14 ventes identifiées, 35 articles et 115,500 TND. Votre valeur ajoutée sera de retrouver le chemin d'une vente et de défendre ces nombres, même après une relance.

DÉPART / 15 HEURES EN PRÉSENCE

Cinq séances, cinq petites victoires

Séance de 3 h	Ce que je peux montrer
1 · Ouvrir la mission	Un dossier prêt, une consigne et un calcul vérifié.
2 · Reconnaître et nettoyer	16 lignes expliquées ; un vrai SELECT source ; une étoile.
3 · Garder les chiffres et l'histoire	Chargement, ajout, rejeu et version de boutique correcte.
4 · Voir et réviser	Quatre CSV, relations et mesure dans Power BI ; un Skill essayé.
5 · Livrer	Un défaut réparé, une revue, une actualisation et mon explication.

Chaque séance comprend 10 min de pause. S1 à S4 : 60 min de concepts et démonstrations courtes + 110 min de manipulation. S5 : 15 min de lancement + 155 min de défi. Total : 255 min de concepts, 595 min de pratique et 50 min de pause. Le temps net est donc 70 % pratique et 30 % concepts.

Un rythme souple

Ces durées sont des estimations de conception, sans pilote humain chronométré. Lecture, réflexion, erreurs et entraide font partie du travail. Aucun devoir n'est nécessaire pour la route principale. Les extensions sont facultatives.

Commencez par 01_demarrer_windows_ubuntu.pdf pour votre poste ; suivez ensuite ce livre dans l'ordre. 03_prompts_reperes_depannage.pdf aide à formuler et dépanner. 04_defi_atlas.pdf décrit S5 ; 05_indices_et_corriges.pdf s'ouvre après une tentative.

S1 / COMPRENDRE AVANT LES OUTILS

BI, ETL et agent : qui fait quoi ?

La BI, Business Intelligence, aide à transformer des données en informations utilisables. Ici : répondre à « combien avons-nous vendu et dans quelle zone ? ». Un montant de ventes est un chiffre d'affaires ; il ne donne ni coût, ni marge, ni nombre de clients uniques.

Mot	Image à retenir
Source	Les lignes reçues : un CSV et une base SQL.
ETL	Extract : lire ; Transform : vérifier et calculer ; Load : ranger.
Entrepôt	Un rangement de ventes et de descriptions stables et historiques.
Rapport	Une vue des nombres avec des filtres.
Modèle / agent	Le modèle produit une réponse ; l'agent utilise aussi des outils pour agir.

Python exécute les règles ; SQLite est un moteur de base locale ; SQL est le langage pour questionner ses tables. Power BI Desktop importe les tables et présente les résultats. Codex aide à lire, planifier, écrire et exécuter, selon ses outils et permissions.

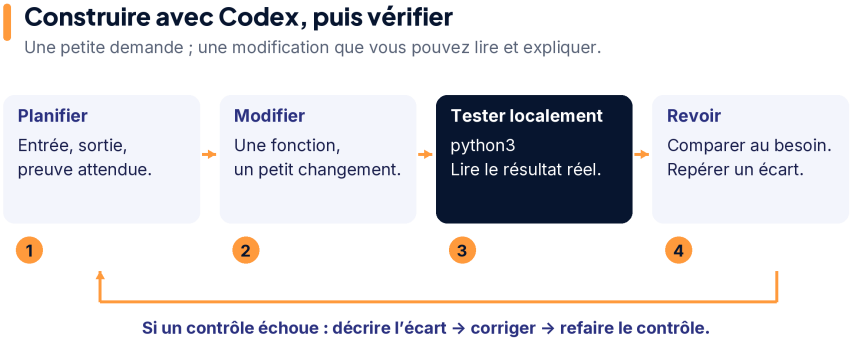
Essayer et expliquer

Classez « lire le CSV », « refuser quantité 0 », « insérer une vente » et « filtrer une zone » dans le chemin. Puis dites : qui calcule le montant réellement exécuté ? Une réponse dans une conversation reste une proposition jusqu'au contrôle.

S1 / UN COLLÈGUE À GUIDER

Codex : du dialogue à une action vérifiée

Codex est un outil OpenAI d'assistance au travail sur le code. Le premier produit Codex annoncé en 2021 aidait déjà à transformer une demande en langage courant en code. La route actuelle du cours utilise l'application de bureau et son espace Codex : lecture de fichiers, petites modifications et commandes locales. Les fonctions dépendent du client disponible.



La proposition devient utile après exécution et comparaison.

Un agent peut lire le mauvais fichier, mal comprendre une règle ou corriger seulement le symptôme. Sa confiance dans une phrase ne mesure pas la justesse du résultat. Vous gardez la décision métier : on rejette une ligne invalide, on n'invente pas une quantité pour faire passer le contrôle.

Première demande, en lecture seule

« Lis le fichier atelier/data/ventes_initiales.csv. Décris une ligne et les colonnes. Propose trois étapes pour comprendre les données. Ne modifie aucun fichier. Sépare ce que tu observes de ce que tu supposes. »

Comparez la réponse au fichier ouvert. Corrigez une phrase imprécise dans votre journal. Sources officielles : https://openai.com/index/openai-codex/ et https://learn.chatgpt.com/docs/app. Les informations d'accès du cours ont été vérifiées dans la documentation le 27 octobre 2026.

S1 / 40 MIN DE MANIPULATION DU POSTE

Mon poste et mes trois espaces

Espace	Ce que j'y fais
Explorateur de fichiers	Ouvrir le dossier, voir les CSV, retrouver les résultats.
Éditeur nano	Lire et enregistrer un fichier texte dans le terminal Ubuntu / WSL : AGENTS.md, SUIVI.md ou un Skill.
Terminal Ubuntu / WSL	Lancer les commandes de ce livre. Ne pas les taper dans du Python interactif.
Conversation Codex	Donner la mission, lire le plan et les changements.

Suivez votre route du livre 01. Windows : application hôte, agent WSL2 et terminal Ubuntu ; Power BI reste sur Windows. Ubuntu : application Linux prise en charge et ETL natif, puis poste Windows en S4. Dans Codex, ouvrez le dossier complet du cours, ~/cours-bi : il contient AGENTS.md et .agents/. Vos fichiers de travail sont dans son sous-dossier atelier/ ; les commandes pratiques se lancent depuis ce sous-dossier.

Terminal Ubuntu / WSL · dans le dossier téléchargé

```
pwd
python3 --version
```

Preuve et secours

Montrez le dossier courant et la version Python. Si python3 est introuvable ou si Codex ne s'ouvre pas, reprenez le livre 01. Pendant la résolution, travaillez en binôme sur un poste fonctionnel de la salle ; consignez que votre propre installation reste à terminer.

Gardez 40 min pour cette étape. L'éditeur ne lance pas votre fichier parce que vous l'avez enregistré ; le terminal attend une commande explicite. Un chemin est l'adresse d'un fichier, pas son contenu.

S1 / JALON 01 · POINT DE DÉPART

Retrouver mon atelier sans recréer ma copie

Le livre 01 a déjà créé atelier/ dans le dossier du cours. Continuez cette copie. Dans le terminal, regardez pwd : si le chemin finit par /atelier, vous êtes prêts. S'il finit par /cours-bi, entrez cd atelier. Gardez le dossier complet ~/cours-bi ouvert dans Codex pour ses règles et ses Skills.

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 01
```

Seulement si atelier/ n'existe pas encore : depuis la racine du cours, lancez python3 outils/jalon.py 01 atelier, puis cd atelier. La création prépare du code, sans exécuter les chargements. Si une copie existe, continuez-la ; gardez les reprises sous un nouveau nom et entrez ce nom exact dans le terminal.

Je vois	Je comprends
data/	Les ventes et catalogues reçus ; source_operationnelle.sqlite est une source SQL.
*.py	Des instructions Python, dont des petites fonctions à compléter.
outputs/	Les reçus, la base construite et les exports produits par exécution.
verifier.py / executer.py	Contrôler une étape / exécuter une action prévue.

À ce stade, je sais...

Retrouver atelier, distinguer fichier et dossier, et montrer où taper une commande. Lisez le contrôle réel : un message TODO désigne une étape à compléter ; un contrôle réussi ne prouve pas encore tout le projet.

S1 / MON ÉDITEUR TEXTE GRATUIT

Modifier et enregistrer un fichier avec nano

nano est un éditeur de texte dans le terminal Ubuntu / WSL. Il permet d'écrire et sauvegarder un fichier ; il n'exécute pas le Python. Gardez Codex ouvert sur le dossier du cours. Dans le terminal, ouvrez le fichier de règles :

Terminal Ubuntu / WSL · ouvrir les règles du cours

```
cd ~/cours-bi
nano AGENTS.md
```

Utilisez les flèches pour placer le curseur. Tapez une règle personnelle, par exemple « Indique l'unité de chaque montant ». Pour enregistrer : maintenez Ctrl et appuyez sur O, la lettre O. Appuyez sur Entrée pour garder le même nom. Puis Ctrl+X quitte nano.

Terminal Ubuntu / WSL · après avoir enregistré et quitté nano

```
cat AGENTS.md
cd atelier
```

Lire le fichier sauvegardé

cat affiche le contenu. Retrouvez votre règle : vous avez vérifié l'enregistrement, sans exécuter le fichier. Les commandes Python du cours se lancent ensuite depuis atelier/. Pour éditer un autre texte, nano suivi de son nom utilise les mêmes touches.

Si nano est introuvable, depuis le terminal Ubuntu / WSL, installez-le avec les commandes suivantes. sudo peut demander votre mot de passe du poste ; aucun caractère n'apparaît pendant sa saisie. Reprenez ensuite nano AGENTS.md depuis ~/cours-bi.

Terminal Ubuntu / WSL · seulement si nano manque

```
sudo apt update
sudo apt install nano
```

Aide officielle : https://www.nano-editor.org/dist/latest/nano.html. En bas de nano, ^O signifie Ctrl+O et ^X signifie Ctrl+X. Le fichier reste modifiable après sa sauvegarde.

S1 / AGENTS.MD ET PLAN

Écrire une instruction qui reste dans le dossier

AGENTS.md contient les règles du projet. Il se trouve à la racine du cours : ~/cours-bi/AGENTS.md, à côté de atelier/. Ouvrez-le dans nano avec les gestes de la page « Modifier et enregistrer un fichier avec nano ». Codex reste ouvert sur ~/cours-bi. Les règles ne remplacent pas votre demande précise.

Texte à adapter · AGENTS.md à la racine du cours

```
Projet : Atlas Papeterie, données fictives.
Explique en français simple avant de modifier.
Préserve atelier/data/ et les noms des fonctions.
Change seulement la fonction demandée.
Garde les montants en millimes entiers.
Montre la commande et le résultat du contrôle.
```

Ajoutez une règle personnelle. Enregistrez : Ctrl+O (lettre O), Entrée pour garder AGENTS.md, puis Ctrl+X. Affichez cat AGENTS.md pour vérifier la règle sauvegardée, puis cd atelier pour retrouver le dossier des commandes Python.

Ma demande

« Lis AGENTS.md. Pour comprendre la première vente, propose trois étapes courtes. Ne change aucun fichier avant de m'avoir présenté le plan. Indique quel contrôle sera utile. »

Un plan utile décrit des actions observables : lire les colonnes, calculer un exemple, comparer au contrôle. « Tout optimiser » ne dit ni quoi ni quand s'arrêter. Choisissez une seule prochaine action avec Codex, puis regardez les fichiers réellement modifiés.

Variation et preuve

Ajoutez une règle personnelle précise : par exemple « écrire l'unité à côté de chaque total ». Ouvrez une nouvelle conversation et demandez à Codex de citer la règle lue. Gardez le fichier et le plan. Source : https://learn.chatgpt.com/docs/agent-configuration/agents-md.

S1 / PRÉVOIR AVANT D'EXÉCUTER

Deux carnets, un premier résultat

Une ligne de vente décrit une vente identifiée, d'un produit dans une boutique à une date. La quantité indique combien d'articles identiques. Ce n'est pas forcément un client ou un ticket complet.

Quantité	Prix unitaire	Montant de la vente
2 carnets	4,500 TND = 4 500 millimes	2 × 4 500 = 9 000 millimes = 9,000 TND

En Tunisie, 1 TND = 1 000 millimes. Dans les fichiers, le point est le séparateur décimal : 4.500. Dans le programme, nous gardons 4500 millimes entiers. Additionner des entiers évite les petites approximations d'un nombre à virgule binaire.

Terminal Ubuntu / WSL · depuis atelier

```
python3 calcul.py
python3 -c "print(2 * 4500)"
```

À essayer

Prédisez 3 carnets au même prix. Changez seulement le 2 dans la commande. Puis expliquez pourquoi 13500 désigne des millimes et pourquoi ce résultat n'est pas 13 500 TND.

Une fonction est une petite recette avec une entrée et une sortie. Plus tard, normaliser_vente recevra une ligne et renverra une ligne contrôlée. Le nom, les arguments et les clés de sortie sont son interface : les autres parties du programme en dépendent.

S1 / OBSERVER SON CLIENT RÉEL

Modèle, réflexion, contexte et usage

La route Free annoncée le 27 octobre 2026 concerne GPT-6 Luna à vitesse Standard dans l'application de bureau, sous réserve de déploiement. Vérifiez ce qui apparaît sur votre compte. Aucun CLI ni clé API n'est requis. Si l'accès Codex gratuit manque, signalez l'écran et poursuivez les tâches locales pendant la résolution. High est le point de départ conseillé pour Luna si votre interface le propose ; sinon gardez le réglage affiché et découpez la tâche. Luna ne propose pas Ultra.

Mot	Ce qu'il signifie ici
Modèle	Le moteur effectivement affiché dans votre client.
Réflexion / effort	Le temps de raisonnement demandé ; ne changez que les options présentes.
Standard	Une vitesse de service ; ce n'est pas le niveau de réflexion.
Contexte	Messages, instructions, fichiers lus et résultats utiles à cette conversation.
Quota	La limite d'usage du compte ; elle est distincte du contexte.

Un token est un morceau de texte ; un mot peut en contenir plusieurs. La taille du prompt ne suffit pas à prédire l'usage. Modèle, difficulté, historique, réflexion et outils comptent. Le cours ne promet pas trois heures continues d'agent par séance.

Petit relevé

Avant puis après une tâche : date, modèle, effort visible ou « non proposé », indicateur d'usage ou « non affiché », réinitialisation affichée et résultat. Ne transformez pas un pourcentage de limite en pourcentage de tokens. Sources : https://learn.chatgpt.com/docs/pricing et https://learn.chatgpt.com/docs/models.

S1 / FIN DE SÉANCE

Garder une preuve et reprendre demain

Une preuve relie attente et observation. « Codex dit que cela fonctionne » ne montre pas l'exécution. Gardez commande, sortie, explication et prochaine action dans ~/cours-bi/SUIVI.md, à la racine du cours. Depuis le terminal Ubuntu / WSL :

Terminal Ubuntu / WSL · créer ou rouvrir le journal à la racine

```
cd ~/cours-bi
nano SUIVI.md
```

Exemple de note · à adapter à ce qui a été observé

```
Mission : lire et expliquer une vente.
Prédiction : 2 x 4500 = 9000 millimes.
Fichiers lus : data/ventes_initiales.csv.
Commande : python3 -c "print(2 * 4500)"
Observé : 9000 ; accord avec ma prédiction.
Prochaine action : classer les lignes douteuses.
```

Si le fichier est nouveau, saisissez votre note sur le modèle ci-dessus ; sinon ajoutez la suite. Ctrl+O, Entrée, Ctrl+X sauvegardent et quittent. Tapez cat SUIVI.md : retrouvez votre dernière phrase. Puis cd atelier ; les prochaines commandes Python se lancent depuis ce sous-dossier.

Même petit problème et dernier résultat utile : continuez la conversation. Nouvelle mission ou nouvelle séance : repartez avec cette note et les fichiers utiles. Si le client propose résumer ou compacter, contrôlez que les décisions restent correctes. Un résumé ne remet pas le quota à zéro.

Si la limite apparaît

Notez la réinitialisation réellement affichée. Continuez à lire les données, calculer, exécuter les scripts locaux et préparer une correction courte. Ce travail fait progresser ; le journal distingue encore la pratique Codex à reprendre.

À ce stade, je sais...

Montrer mon atelier, donner un objectif et une limite à Codex, prédire un montant, exécuter une commande et conserver une preuve. Sans regarder la réponse, expliquez à un camarade la différence entre agent, terminal et résultat.

S2 / JALON 02 · QUALITÉ

Chaque ligne reçue doit avoir une destination

La direction ne peut pas additionner directement le fichier. Une répétition gonfle le montant ; une quantité impossible fausse les chiffres. Nettoyer consiste à appliquer des règles explicites, puis à garder la trace des lignes non retenues.

Exemple	Décision à prévoir
« nbe » avec des espaces	Retirer les espaces et mettre en majuscules.
Même sale_id, mêmes valeurs normalisées	Garder une vente ; compter la répétition comme doublon.
Quantité 0 ou date impossible	Rejeter avec un motif compréhensible.
Même sale_id, valeurs différentes	Signaler un conflit ; ne pas écraser une vente en silence.

Une ligne lue a une issue explicite

Lot initial · chiffres attendus du contrat · les sources restent intactes.

16
lignes lues
Le fichier initial.

12
ventes retenues
Valides et distinctes.

1
doublon identique
Déjà vu, même contenu.

3
lignes rejetées
Quantité, produit ou date invalide.

16 = 12 + 1 + 3

Un doublon différent portant le même sale_id est un conflit : on arrête et on explique.

Après chargement initial : 12 faits · 32 unités · 99 000 millimes.

Une destination pour chacune des 16 lignes reçues.

À essayer avant le code

Avant le code, ouvrez data/ventes_initiales.csv. Repérez une ligne douteuse et notez sa décision sans modifier la source. Deux ventes du même produit à des dates différentes peuvent être distinctes : regardez sale_id.

Preuve finale attendue : 16 = 12 + 1 + 3. Ce bilan porte sur les lignes reçues, pas sur le nombre d'articles. Les 12 ventes acceptées représentent 32 articles et 99 000 millimes.

S2 / UNE MODIFICATION LIMITÉE

Lire le Python d’une petite fonction

Un dictionnaire associe une clé à une valeur. ligne["shop_id"] désigne la valeur de la colonne shop_id. Le point appelle une opération du texte. strip() retire les espaces au bord ; upper() met les lettres en majuscules.

Python illustratif · à lire avant de modifier

```
code_boutique = " nbe "
propre = code_boutique.strip().upper()
print(propre)
```

La sortie attendue est NBE. Cette opération ne décide pas si le code existe : le catalogue boutiques.csv joue ce rôle. Une condition permet ensuite d’accepter ou de refuser. Une erreur explicite aide à garder la raison du refus.

Petite mission Codex

« Lis l’amorce normaliser_vente de atelier/nettoyage.py et les fonctions d’aide fournies. Complète seulement cette fonction : identifiants normalisés, date réelle, quantité entière positive, prix exact en millimes. Préserve son interface. Donne le contrôle à lancer et explique une ligne modifiée. »

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 02
```

Variation et preuve

Avant de lancer, prévoyez le sort de « tun » et de quantité « 0 ». Après le contrôle, expliquez une ligne de votre fonction avec vos mots. Si vous ne pouvez pas la lire, demandez une explication sur cette seule ligne ; n’acceptez pas une deuxième réécriture globale.

S2 / UNE UNITÉ, DES RÈGLES

Pourquoi garder les millimes entiers ?

Entrée texte	Valeur utile	Décision
4.500	4500 millimes	Prix accepté.
1.250	1250 millimes	Prix accepté.
0 ou négatif	Aucune	Prix rejeté.
0.0001	Un dixième de millime	Rejeté : ne pas arrondir en silence.

Le module Decimal lit le prix comme un texte décimal exact. Le programme vérifie que le prix est fini, positif et que sa multiplication par 1 000 donne un entier exact. Le helper fourni millimes applique ce contrat. La quantité est un entier positif ; 2.5 articles n'est pas accepté.

Calcul puis vérification

Pour quantité 3 et prix « 1.250 », prévoyez unit_price_millimes et amount_millimes. Le premier vaut 1 250 ; le second 3 × 1 250 = 3 750. Le montant dépend de la ligne, pas d'un total final inscrit dans le code.

Dans la sortie normalisée : quantity, unit_price_millimes et amount_millimes sont des entiers. Les noms anglais sont conservés parce que les fichiers et les autres fonctions s'accordent sur ces noms.

Si je suis bloqué

Une conversion de texte échoue : regardez la valeur exacte, ses espaces et le séparateur. Une virgule n'est pas un point. Ne corrigez pas la source pour cacher le problème ; laissez le programme produire un motif et gardez le bilan.

S2 / DES COMPTES ET DES MOTIFS

Comparer mon nettoyage à la règle métier

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 02
```

Le vérificateur essaie des exemples. Lisez son résultat et la fonction modifiée. Un test réussi est une preuve sur les cas contrôlés ; il ne rend pas votre fonction infaillible. Cherchez aussi un exemple proche différent de celui du prompt.

Question	Ce que je dois expliquer
Combien de lignes reçues ?	16, sans l'en-tête.
Combien de ventes retenues ?	12 ; une répétition n'ajoute pas une vente.
Pourquoi 3 rejets ?	Chaque ligne possède un motif de règle violée.
Quel total accepté ?	32 articles ; 99 000 millimes = 99,000 TND.

Une expérience sans changer les données d'origine

Demandez à Codex de décrire un essai en mémoire pour quantity = « 2 », puis « 0 ». Prédisez les décisions. Exécutez le contrôle approprié et gardez une observation qui aurait révélé une erreur.

Si l'attendu diffère de l'observé : vérifiez d'abord le dossier, le fichier enregistré et l'unité. Puis donnez à Codex votre fonction, la règle violée et le message exact. Après deux corrections sans progrès, revenez au petit exemple avec un camarade.

À ce stade, je sais...

Distinguer harmonisation, doublon et rejet ; lire une petite fonction et justifier le bilan 16 = 12 + 1 + 3. Je peux montrer une décision que le contrôle vérifie réellement.

S2 / JALON 03 · EXTRACTION RÉELLE

Un fichier CSV et une table SQL sont deux sources

Le CSV est un fichier texte : l'en-tête nomme les colonnes, une ligne décrit une vente. Une base SQLite contient des tables dans un fichier. Nous lisons data/source_operationnelle.sqlite ; ce n'est pas l'entrepôt outputs/atlas.sqlite que nous construirons ensuite.

Source	Données de notre mission
ventes_initiales.csv	16 lignes du premier envoi.
source_sales	Le lot SQL : S012, S013 et S014.
source_shop_changes	NBE passe à Grand Tunis le 8 janvier 2026.

SQL · requête à lire dans lecture.py

```
SELECT sale_id, date, shop_id, product_id,
       quantity, unit_price_tnd
FROM source_sales
WHERE batch_id = ?
ORDER BY sale_id
```

SELECT choisit les colonnes ; FROM choisit la table ; WHERE garde le lot ; ORDER BY rend l'ordre lisible. Le point d'interrogation attend un paramètre : « 2026-01-08 ». On transmet cette valeur séparément au moteur SQL.

À prévoir

Combien de ventes sortent du SELECT ? Trois. Combien seront nouvelles dans l'entrepôt ? Deux seulement : S012 reviendra déjà dans l'initial. Extraire et charger désignent deux étapes différentes.

S2 / DE LA REQUÊTE À SA TRACE

Exécuter un SELECT avec son paramètre

Ouvrez lecture.py. La connexion à la source et la trace de lecture sont fournies. Complétez la petite lecture de lot en utilisant les requêtes prévues et un tuple de paramètres. Une virgule permet de former un tuple d'une seule valeur en Python.

Python · motif de lecture paramétrée, contexte fourni

```
lignes = conn.execute(
    SQL_VENTES, (batch_id,)
).fetchall()
```

Demande Codex ciblée

« Complète lire_increment_sql dans atelier/lecture.py. Exécute réellement SQL_VENTES et SQL_CHANGEMENTS sur la connexion source fournie, avec batch_id comme paramètre. Garde les trois sorties prévues : ventes, changements, preuve. Ne remplace pas SQL par un CSV. Explique le rôle du ?. »

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 03
```

Le contrôle doit observer 3 ventes et 1 changement de zone. La preuve d'extraction conserve la requête, le paramètre, le chemin et l'empreinte de la source. L'empreinte aide à savoir quel fichier a été lu ; elle ne prouve pas que les données d'origine sont vraies.

Aide et auto-contrôle

« no such table » : vérifiez le fichier source. « unable to open » : vérifiez son chemin, pas un CSV de secours inventé. Expliquez pourquoi changer une ligne de la vraie source sur une copie d'essai doit changer le résultat extrait.

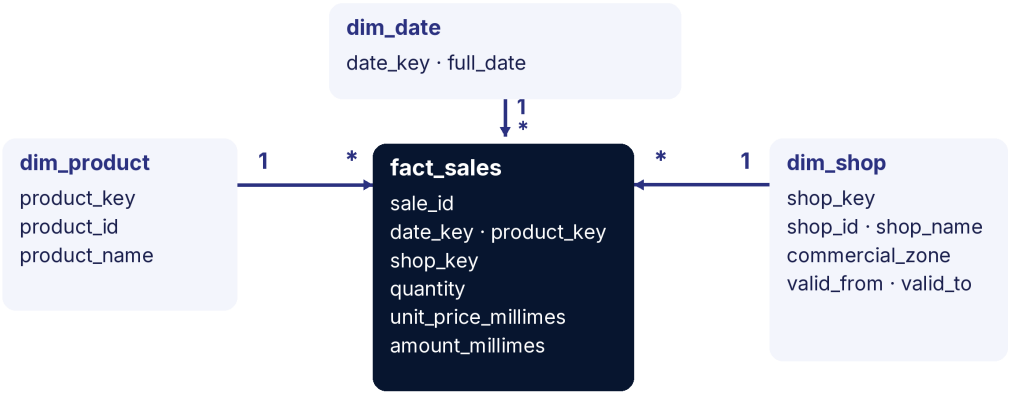
S2 / PREMIÈRE ÉTOILE

Placer la vente au centre de ses descriptions

Une table est un ensemble de lignes avec les mêmes colonnes. La table de faits garde ce qui s'est passé : une vente, sa quantité et son montant. Une dimension décrit cette vente : le produit, le jour et la version de boutique.

L'étoile : décrire chaque vente

Trois relations actives 1 → plusieurs, de la dimension vers les ventes.



shop_id = NBE identifie la boutique ; shop_key identifie une version.

Relier dim_shop.shop_key à fact_sales.shop_key préserve l'histoire.

Une flèche = le sens du filtre ; 1 et * = une clé de dimension pour plusieurs ventes.

Trois dimensions filtrent les faits par leurs clés techniques uniques.

Cartes à classer

Placez amount_millimes et quantity dans Vente. Placez product_name dans Produit, full_date dans Date et commercial_zone dans Boutique. Tracez une flèche d'une vente vers chacune de ses trois descriptions.

Le grain fixe le sens d'une ligne du fait : une vente identifiée par sale_id. Additionner quantity compte des articles ; compter les lignes de fact_sales compte des ventes identifiées. Ces deux nombres sont 35 et 14 à la fin, sans contradiction.

S2 / FIN DE SÉANCE

Deux types de clés, deux usages

Clé	Exemple	Rôle
Métier	shop_id = NBE	Le code de la boutique dans les sources.
Technique	shop_key = un entier	L'identité d'une version dans l'entrepôt.
Étrangère	shop_key dans fact_sales	La référence d'une vente à sa version de boutique.

Un même shop_id aura deux versions après le changement. shop_id ne sera donc plus unique dans dim_shop. Chaque shop_key restera unique. C'est cette clé technique que la vente et Power BI utiliseront pour retrouver une seule description.

À essayer

Dessinez une vente S013 : NBE, 8 janvier, un carnet. Reliez-la à Date, Produit et Boutique. N'inventez pas son numéro de shop_key ; ce numéro sera attribué et observé après chargement.

Préparez S3 : gardez dans SUIVI.md le contrôle 02, le contrôle 03 et une phrase sur la différence CSV/base SQL. Ajoutez le plan de l'étoile. Si une étape bloque encore, consultez l'indice de cette étape dans le livre 05.

À ce stade, je sais...

Lire un CSV, expliquer un SELECT avec son paramètre et classer quantité, montant et descriptions. Je distingue 3 lignes extraites de 2 ventes nouvelles, et un identifiant métier d'une clé de version.

S3 / JALON 04 · PREMIER CHARGEMENT

Charger les descriptions avant les ventes

Une clé étrangère est une règle de lien : la vente ne doit pas pointer vers un produit ou une boutique qui n'existe pas. L'ossature fournie crée les tables et active ce contrôle avant la transaction. Nous inspectons son effet au lieu de reconstruire le moteur.

1. Produits / boutiques
2. Dates
3. Ventes et clés
4. Contrôle puis validation

Terminal Ubuntu / WSL · depuis atelier

```
python3 executer.py initial
python3 verifier.py 04
```

L'action lit le CSV, applique le nettoyage, prépare les dimensions puis insère les faits dans outputs/atlas.sqlite. Un reçu rend visible ce qui a été retenu et chargé. Ouvrez outputs/qualite_initial.json et outputs/chargement_initial.json dans l'éditeur.

À retrouver	Valeur attendue
Faits / quantité	12 ventes / 32 articles
Somme des montants	99 000 millimes
Dimensions	4 produits ; 2 versions de boutique ; 3 dates

À expliquer et à essayer

Pourquoi charger Produit avant Vente ? Dans une copie d'essai uniquement, le contrôle d'une clé absente doit refuser la vente. Si une FK semble ignorée, demandez quelle connexion a activé foreign_keys ; une déclaration de colonne ne suffit pas à prouver le contrôle actif.

S3 / UN CONTRÔLE QUI NE DÉPEND PAS DU GRAPHIQUE

Lire le total dans la base construite

SQLite conserve des tables ; on peut les questionner sans ouvrir Power BI. La requête suivante lit le nombre de faits, la quantité et le montant. Elle agrège : plusieurs lignes deviennent une ligne de résultat.

SQL · requête de contrôle sur outputs/atlas.sqlite

```
SELECT COUNT(*) AS ventes,
       SUM(quantity) AS articles,
       SUM(amount_millimes) AS millimes
FROM fact_sales;
```

Terminal Ubuntu / WSL · depuis atelier

```
python3 executer.py rapport
```

Ouvrez outputs/rapport.json. Comparez ses valeurs au reçu de chargement. Le rapport JSON est une preuve locale de calcul ; le rapport destiné à la direction sera construit dans Power BI Desktop en S4.

Ma prédiction

Avant l'incrément : 12, 32, 99000. Après l'incrément : 14, 35, 115500. La différence sera 2 ventes, 3 articles et 16 500 millimes. Prédisez ces différences avant de lancer le lot.

L'action rapport produit aussi outputs/rapport_atlas.html. Vous pouvez l'ouvrir dans un navigateur pour lire les preuves ETL locales ; ce fichier ne remplace pas le modèle et l'actualisation Power BI sur Windows.

Si je suis bloqué

« no such table » peut désigner la mauvaise base ou un initial non exécuté. « file not found » signifie que l'artefact n'a pas été produit à cet endroit. Une ancienne sortie dans outputs/ n'est pas une preuve du dernier essai : notez l'action exécutée et sa date.

S3 / INCRÉMENT ET REJEU

Ajouter un petit lot sans tout recompter

Vente extraite	Ce qu'on sait	Décision
S012	Déjà chargée ; contenu identique	Déjà présente.
S013	NBE, 8 janvier, 1 × 4.500 TND	Nouvelle vente : 4 500 millimes.
S014	TUN, 8 janvier, 2 × 6.000 TND	Nouvelle vente : 12 000 millimes.

Un chargement incrémental traite un lot complémentaire. Un rejeu traite de nouveau le même lot. Pour rester fiable, même identifiant et mêmes valeurs ne créent pas une deuxième vente. Même identifiant et valeurs différentes provoquent un conflit explicite.

Avant d'exécuter

Le lot contient aussi le changement de zone. Nous devons donc comprendre et compléter la recherche de version sur les pages suivantes avant de lancer increment. Prévoyez le total : 99 000 + 4 500 + 12 000 = 115 500 millimes.

Le moteur fourni rassemble les modifications dans une transaction : un groupe d'actions validées ensemble. En cas de conflit, rollback annule le groupe. Sans cela, une nouvelle version de boutique pourrait rester alors que les ventes ont échoué.

Question

Pourquoi « trois lignes SQL » ne doit-il pas devenir « trois ventes de plus » ? Montrez S012 et dites quelle preuve distingue une répétition d'un conflit.

S3 / JALON 05 · AVANT LE NOM SCD2

Nabeul change de zone, les ventes gardent leur histoire

Nabeul reste la même boutique. Le 8 janvier 2026, son rattachement commercial passe de Cap Bon à Grand Tunis. Il ne s'agit pas d'un déplacement géographique. Nous devons lire la zone valable le jour de chaque vente.



Le 8 janvier est exclu de Cap Bon et inclus dans Grand Tunis ; les anciennes ventes gardent leur version.

Une frontière précise

Une vente NBE du 7 janvier pointe vers Cap Bon. Une vente du 8 janvier pointe vers Grand Tunis. Le 8 n'appartient pas aux deux versions : l'ancienne fin est exclue. On écrit cet intervalle [début, fin).

Cette conservation de plusieurs versions porte le nom SCD2 : Slowly Changing Dimension de type 2. Le mot arrive après l'idée. Vous recevez la mécanique qui ferme l'ancienne version et crée la nouvelle ; votre tâche est d'expliquer et vérifier la recherche de la bonne version.

À prévoir

Dessinez deux cartes NBE. Placez une vente du 7 et une du 8 sur la bonne carte. Puis demandez : si une vente du 7 arrive en retard, quelle carte doit-elle prendre ? La carte historique Cap Bon.

S3 / PETITE FONCTION TEMPORELLE

Chercher exactement une version à la date de vente

SQL · SQL_VERSION fourni dans incremental.py

```
SELECT shop_key FROM dim_shop
WHERE shop_id = ? AND valid_from <= ?
  AND (valid_to IS NULL OR ? < valid_to)
```

Les trois paramètres sont le code de boutique puis deux fois la date de vente. « valid_from <= date » inclut le début. « date < valid_to » exclut la fin. IS NULL signifie que la borne finale est encore ouverte.

Petite mission Codex

« Complète seulement trouver_shop_key(conn, shop_id, sale_date) dans atelier/incremental.py avec SQL_VERSION. La requête reçoit trois paramètres. Il faut exactement un résultat : zéro ou plusieurs signalent une erreur. Retourne son shop_key. Ne change pas les anciennes ventes. Explique la frontière du 8 janvier. »

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 05
```

Ne choisissez pas simplement la version actuelle : cela reclasserait les ventes anciennes reçues en retard. Ne prenez pas le premier résultat s'il y en a plusieurs : un chevauchement est une erreur d'histoire à révéler.

Preuve et aide

Montrez le contrôle du 7 puis du 8, et dites pourquoi le signe final est <. Si le contrôle 05 échoue, regardez d'abord l'ordre des paramètres et le nombre de lignes trouvées. Les extensions rétroactives générales sont hors parcours.

S3 / JALON 06 · LES PREUVES D'HISTOIRE

Lancer, relancer et comparer les clés

Terminal Ubuntu / WSL · depuis atelier

```
python3 executer.py increment
python3 executer.py rapport
python3 verifier.py 06
```

Avant de continuer, ouvrez outputs/chargement_increment.json. Notez dans SUIVI.md le premier résultat : 2 ventes nouvelles, 1 déjà présente et 1 nouvelle version. Gardez aussi les valeurs et clés observées. Le prochain lancement remplace le reçu.

Terminal Ubuntu / WSL · depuis atelier · même lot rejoué

```
python3 executer.py increment
python3 executer.py rapport
```

Observation	Premier lot	Même lot rejoué
Nouvelles ventes	2	0
Ventes déjà présentes	1	3
Nouvelles versions NBE	1	0
Total final	14 faits ; 35 articles ; 115 500 millimes	Identique

Ouvrez outputs/extraction_sql.json et le nouveau outputs/chargement_increment.json. Comparez au premier résultat conservé : aucune insertion, 3 ventes déjà présentes et aucune nouvelle version. Le contrôle compare aussi les anciennes clés et valeurs ; le total seul ne suffirait pas.

À ce stade, je sais...

Distinguer deux boutiques de trois versions, choisir la version selon la date et expliquer un rejeu. Montrez qu'une ancienne vente conserve son shop_key et que le 8 janvier ne couvre qu'une seule version NBE.

S3 / CONFIANCE GAGNÉE PAR DES PREUVES

Faire relire puis contrôler la correction

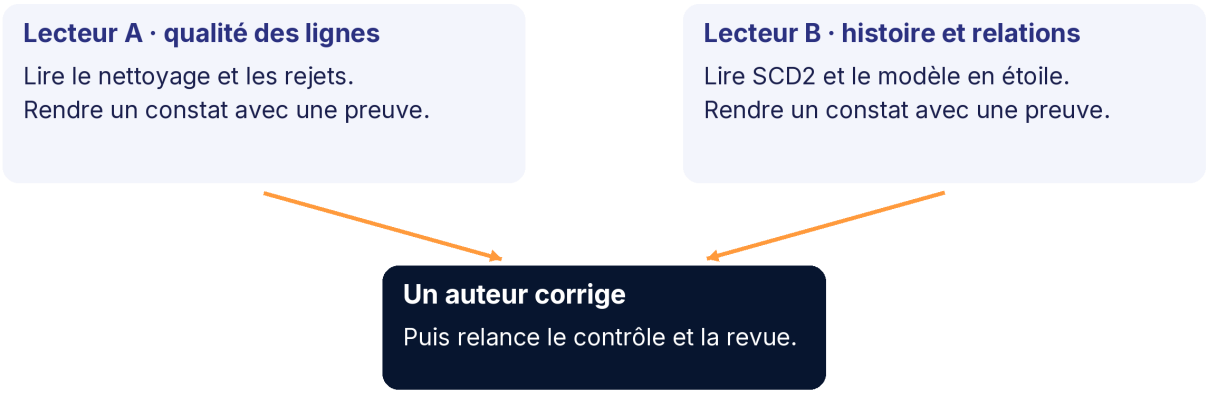
Une revue cherche un défaut établi sur une petite cible. Donnez la règle et les fichiers ; demandez les preuves et les limites non vérifiées.

Demande de revue

« Lis atelier/incremental.py et les preuves du contrôle 06, sans écriture. Vérifie la borne du 8 janvier et les anciennes clés. Cite règle, fichier et exemple pour chaque défaut établi ; sinon indique la preuve manquante. »

Deux revues indépendantes, un seul auteur

Au plus deux lecteurs sur des cibles distinctes ; personne ne modifie en parallèle.



Si la fonction manque sur Free : deux passes séparées, ou deux revues humaines.

Une nouvelle lecture porte sur le résultat corrigé.

Choisissez une correction justifiée et un seul auteur. Relancez le contrôle concerné, puis demandez une deuxième lecture du résultat corrigé. Un test vert avant la correction n’est pas un contrôle de la version modifiée.

Variation

Si aucun défaut n’est trouvé, discutez une vente tardive du 7 janvier. « Pas de défaut trouvé » décrit une lecture ; n’inventez pas une erreur pour remplir le journal.

Fin de S3

Dans SUIVI.md, gardez total, trois versions, résultat du rejeu et prochaine action : exporter les quatre tables. Vous êtes prêts à rendre les données visibles sans perdre l’histoire.

S3 / LE RÉSULTAT VISIBLE DE MON ETL

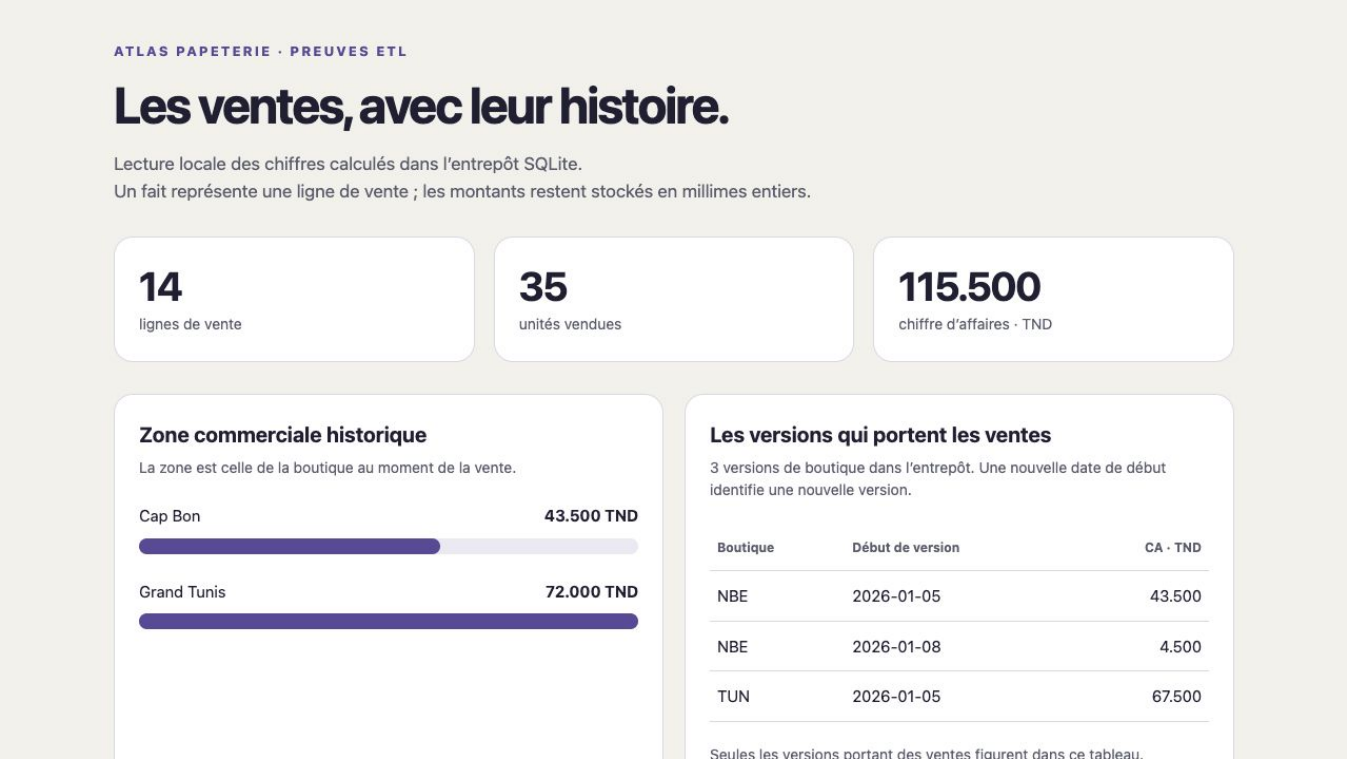
Voir les preuves locales avant Power BI

Après le chargement et le rejeu, l'action rapport produit deux lectures des mêmes résultats : rapport.json et rapport_atlas.html dans outputs/. Le HTML se consulte dans un navigateur à partir du fichier local ; aucun service supplémentaire n'est nécessaire.

Terminal Ubuntu / WSL · depuis atelier · après l'incrément

```
python3 executer.py rapport
```

Dans l'explorateur de fichiers, ouvrez atelier/outputs/rapport_atlas.html avec votre navigateur. Retrouvez les trois indicateurs puis les zones historiques. Comparez-les à votre propre reçu et au rapport JSON.



Capture réelle du rapport HTML local, auteur sur macOS, données Atlas : preuve ETL. Power BI reste à réaliser sur Windows. Cette capture ne prouve aucun accès Free ni installation WSL.

À essayer et à expliquer

Montrez pourquoi 14 ventes représentent 35 articles. Retrouvez 43,500 TND pour Cap Bon et 72,000 TND pour Grand Tunis. Expliquez les deux lignes NBE : la même boutique possède deux versions à des dates différentes.

Ce rapport rend vos preuves ETL faciles à lire. La séance suivante construit le modèle Power BI Windows : import des quatre tables, relations, mesure, filtre et actualisation. Vous comparerez les résultats aux mêmes contrôles locaux.

S4 / JALON 07 · EXPORT

Quatre tables voyagent ensemble

Power BI lira un instantané complet : toutes les lignes des quatre tables, y compris les anciennes versions de boutique. Il ne reçoit pas un total précalculé ni une seule table aplatie. Les clés gardent les liens entre les fichiers.

Terminal Ubuntu / WSL · depuis atelier

```
python3 executer.py export
python3 verifier.py 07
```

Fichier du dossier annoncé	Contenu
dim_product.csv	4 produits et leurs clés.
dim_shop.csv	3 versions de boutique avec bornes de validité.
dim_date.csv	4 dates.
fact_sales.csv	14 ventes et leurs trois clés de dimension.
manifest.json	Comptages, sommes et empreintes des quatre fichiers.

Le terminal annonce un dossier outputs/powerbi/<delivery_id>/. <delivery_id> désigne le nom réellement produit ; ce n'est pas un texte à recopier. Gardez ces cinq fichiers ensemble. Le manifeste apparaît après un export complet.

À essayer et à prouver

Ouvrez les quatre en-têtes dans l'éditeur. Repérez shop_key dans la dimension et dans les faits. Le manifeste aide à détecter un mélange de livraisons ; il ne remplace pas le contrôle des relations et des chiffres dans Power BI.

S4 / UN TRANSFERT RÉEL VERS WINDOWS

Passer de WSL ou Ubuntu au poste Power BI

Power BI Desktop s'exécute sur Windows. Sur Windows/WSL, placez-vous dans le dossier d'export annoncé puis ouvrez-le dans l'Explorateur. Copiez les quatre CSV et le manifeste vers un dossier Windows local choisi, par exemple Documents/Atlas-PowerBI.

Terminal WSL · depuis le dossier d'export complet

```
explorer.exe .
```

Sur Ubuntu natif, transférez le même paquet complet vers un poste Windows de la salle ou d'un camarade, par clé USB ou partage de fichiers disponible. Gardez une note du dossier source, du dossier Windows et de la livraison copiée. Le binôme réalise ensuite les manipulations dans Power BI Desktop Windows.

Vérifier avant d'importer

Sur le poste Windows, montrez les cinq fichiers du même export. Ne laissez pas Power BI pointer vers un dossier temporaire qui disparaîtra. Conservez aussi le dossier pour l'actualisation en S5.

Un rapport JSON ou HTML local peut montrer les preuves ETL pendant un blocage ; il ne valide pas l'exercice Power BI. La route du cours ne suppose ni compte de publication cloud, ni exécution de Desktop dans Ubuntu.

Auto-contrôle

Je peux dire où tourne Python, où se trouvent les CSV importés et où tourne Power BI. Une revue de procédure et une manipulation réellement faite sur Windows sont notées séparément dans mon journal.

S4 / POWER BI DESKTOP · WINDOWS

Importer les fichiers et vérifier leurs types

Ouvrez Power BI Desktop. Accueil → Obtenir des données → Texte/CSV. Choisissez fact_sales.csv dans votre dossier Windows, vérifiez délimiteur virgule et encodage UTF-8, puis choisissez Transformer les données pour inspecter les colonnes. Répétez pour les trois dimensions.

Colonnes	Type attendu
*_key, quantity, *_millimes	Nombre entier.
sale_id, shop_id, product_id, noms et zone	Texte.
full_date, valid_from, valid_to	Date ; valid_to vide devient null.
year, month, day	Nombre entier.

Dans Power Query, cliquez l'icône de type en tête de colonne. Si une date ISO n'est pas reconnue, choisissez Date avec paramètres régionaux adaptés et vérifiez le 8 janvier. Les montants sont des entiers : n'appliquez pas une conversion monétaire avant la mesure.

Mon contrôle

Fermer et appliquer. Dans la vue Données, retrouvez 14 faits et 3 lignes dim_shop. Deux lignes NBE sont attendues. Une alerte de doublon sur shop_id ne se règle pas en supprimant l'ancienne version.

Aide

Un total multiplié par 1 000 indique souvent une unité mal lue. Des erreurs de type demandent d'inspecter la colonne et son étape de conversion. Guide Microsoft : https://learn.microsoft.com/power-bi/connect-data/desktop-data-sources.

S4 / POWER BI · VUE MODÈLE

Construire trois relations qui gardent le grain

Ouvrez la vue Modèle. Reliez chaque clé de dimension à la même clé dans fact_sales. Une dimension possède une ligne par clé ; plusieurs ventes peuvent utiliser cette clé. Choisissez cardinalité un à plusieurs (1:*) et filtrage simple depuis la dimension.

Côté 1	Côté *
dim_product.product_key	fact_sales.product_key
dim_shop.shop_key	fact_sales.shop_key
dim_date.date_key	fact_sales.date_key

La relation décisive

Utilisez shop_key, pas shop_id. dim_shop a deux lignes NBE. Joindre les faits au seul code NBE risquerait de leur attribuer plusieurs zones ou de perdre l'histoire.

Si Power BI a deviné une autre relation, inspectez-la avant de la garder. Dans Gérer les relations, vérifiez les deux colonnes, 1:*, le sens simple et l'état actif. Le modèle final comporte trois relations actives prévues.

À essayer et à montrer

Sélectionnez une relation et expliquez quel côté peut avoir plusieurs lignes. Prenez une capture du modèle réel. Si le lien devient plusieurs-à-plusieurs, vérifiez que vous avez choisi la clé technique unique ; ne forcez pas le réglage pour masquer le problème.

Source : https://learn.microsoft.com/power-bi/transform-model/desktop-relationships-understand. Les menus peuvent varier ; le critère reste la même relation vérifiable.

S4 / POWER BI · DES NOMBRES À COMPARER

Une mesure, une carte et un graphique lisible

Une mesure calcule un résultat pour les filtres actifs. Sélectionnez fact_sales, puis Nouvelle mesure. Entrez la formule fournie : elle additionne les millimes et les divise par 1 000 pour afficher des TND.

DAX · Nouvelle mesure dans Power BI

```
CA TND =
DIVIDE(SUM(fact_sales[amount_millimes]), 1000)
```

Revenez à la vue Rapport : icône de rapport dans la barre de gauche. Cliquez une zone vide. Dans Visualisations → Générer un visuel (ou Construire le visuel selon la version), choisissez Carte. Depuis Données, glissez CA TND dans Valeurs, ou le champ équivalent proposé. Dans Format du visuel → Visuel → Valeurs de légende → Valeurs, réglez les unités sur Aucune et les décimales sur 3 ; les noms peuvent varier selon la version.

Cliquez de nouveau une zone vide. Choisissez Histogramme groupé dans Visualisations. Glissez dim_shop[commercial_zone] vers Axe X, puis CA TND vers Axe Y / Valeurs. Donnez le titre « Ventes par zone à la date de vente — TND ». Les barres décrivent des ventes, pas un bénéfice ou des personnes.

Zone historique	Attendu
Cap Bon	43,500 TND
Grand Tunis	72,000 TND
Total sans filtre	115,500 TND

Variation et preuve

Ajoutez un segment sur shop_name. Choisissez Nabeul : attendez 48,000 TND, répartis 43,500 avant changement et 4,500 après. Effacez le filtre. Comparez au contrôle local avant d'enregistrer Atlas.pbix.

S4 / POWER BI · UNE REPRISE PRÉPARÉE

Actualiser et contrôler ce qui vient de changer

Enregistrez le rapport .pbix dans votre dossier Windows. Dans votre journal, notez les chemins des quatre CSV. Power BI relit ces chemins lors de l'actualisation ; une nouvelle livraison placée ailleurs ne remplace pas automatiquement la précédente.

- Reproduire export après un contrôle local réussi.
- Copier ensemble les quatre nouveaux CSV et le manifeste dans le dossier Windows déjà utilisé, ou modifier explicitement les quatre chemins source.
- Dans Power BI, cliquer Actualiser puis vérifier le modèle, la carte et les zones.
- Essayer un filtre connu, l'effacer puis enregistrer le rapport.

Preuves à conserver

Une capture des trois relations actives et une capture du total/graphique ; la livraison importée ; la valeur après actualisation. Notez « Windows exécuté » uniquement si ces actions ont été réellement faites sur ce poste.

Un filtre peut expliquer une différence : regardez segments, filtres de page et sélections de graphique. Une carte juste ne prouve pas une relation juste ; les totaux par zone servent à vérifier l'histoire.

À ce stade, je sais...

Importer les quatre tables, fixer les types, créer les trois relations, lire la mesure et rapprocher un chiffre de SQL. Je peux montrer un filtre et revenir au total, puis actualiser le même rapport.

S4 / LA RECETTE QUE JE PEUX RÉUTILISER

Enregistrer une méthode dans un Skill local

Un Skill garde une méthode dans un fichier SKILL.md. name et description indiquent son nom et son usage. Les deux Skills fournis servent de modèles ; créez votre propre copie locale. Aucun outil, modèle ni quota supplémentaire n'est ajouté.

Fichier à ouvrir	Utilité
.agents/skills/expliquer-bi/SKILL.md	Un exemple concret, une unité, puis une question pour réfléchir.
.agents/skills/revoir-bi/SKILL.md	Une lecture sans correction, avec règle et preuve.

Depuis le terminal Ubuntu / WSL, revenez à la racine du cours. Créez le dossier, copiez le modèle, puis ouvrez votre copie :

Terminal Ubuntu / WSL · copie locale puis édition

```
cd ~/cours-bi
mkdir -p .agents/skills/ma-revue-bi
cp .agents/skills/revoir-bi/SKILL.md \
  .agents/skills/ma-revue-bi/SKILL.md
nano .agents/skills/ma-revue-bi/SKILL.md
```

Avec les flèches, changez name: revoir-bi en name: ma-revue-bi. Ajoutez la règle « demander la preuve du 7 et du 8 janvier avant de conclure ». Ctrl+O, Entrée, Ctrl+X. Vérifiez avec cat .agents/skills/ma-revue-bi/SKILL.md. Puis cd atelier. Essayez la découverte dans le sélecteur ; une nouvelle conversation peut être nécessaire.

Prompt · si l'invocation par nom est proposée

```
Utilise $ma-revue-bi pour relire
la recherche de version de boutique.
```

Autre manipulation honnête

Si le Skill n'apparaît pas : « Lis .agents/skills/ma-revue-bi/SKILL.md et applique sa méthode. » Cela vérifie la lecture explicite, pas la découverte automatique. Gardez fichier et réponse appliquant votre règle.

Source : https://learn.chatgpt.com/docs/build-skills. Si le dossier est masqué dans l'explorateur, activez l'affichage des fichiers cachés ou ouvrez le chemin dans l'éditeur.

S4 / REVUE · CORRECTION · DEUXIÈME REVUE

Deux lectures utiles, puis un seul auteur

Utilisez votre ma-revue-bi sur une petite cible ; revoir-bi reste le modèle fourni. La revue ne modifie rien. Le travail principal applique les corrections après avoir reçu les observations.

Lecture A · données	Lecture B · modèle
Bilan 16 = 12 + 1 + 3 ; unités ; identité ; rejets.	Grain ; clés ; frontière SCD2 ; trois relations ; zones.

Parallèle, si disponible

« Délègue au maximum deux lectures sans écriture : A vérifie la qualité et les nombres ; B vérifie l'histoire et les clés. Attends les résultats ; ne corrige rien. Rassemble défauts, preuves et contrôles manquants. » Observez l'activité réellement créée et notez son coût d'usage s'il apparaît.

Si les sous-agents ne sont pas disponibles ou si l'usage est trop bas, faites deux passes successives dans des conversations séparées, ou partagez une lecture avec un camarade. Notez « deux passes successives » ; jouer deux rôles dans une réponse ne crée pas deux agents indépendants.

La boucle complète

Choisissez un défaut établi → un seul auteur corrige → contrôle local concerné → deuxième revue du résultat modifié. Conservez ce qui a changé et pourquoi. Source sous-agents : https://learn.chatgpt.com/docs/agent-configuration/subagents.

Fin de S4

Votre preuve : export complet, Power BI Windows manipulé, Skill modifié puis essayé, revue et deuxième lecture. En S5, vous livrez le même Atlas avec une petite panne à réparer.

S5 / DÉFI · 2 H 35 DE PRATIQUE

Livrer Atlas et expliquer ma contribution

Ouvrez 04_defi_atlas.pdf. Par groupe de un à trois, vous recevez une copie isolée avec un petit défaut. Vous connaissez déjà le scénario et le rapport ; le défi demande de diagnostiquer, corriger et prouver, sans ajouter un deuxième domaine métier.

Cinq séances, cinq victoires visibles

5 × 3 h = 15 h, pauses comprises · un seul projet Atlas Papeterie.



Chaque semaine : demander petit → exécuter → vérifier → expliquer.

Cinq séances, un même projet et des preuves qui grandissent.

Chacun garde une contribution montrable : fichier ou manipulation, prédiction, contrôle exécuté et explication. À deux ou seul, fusionnez les rôles. N'attendez pas qu'une personne fasse tout le code et tout le rapport.

Votre dossier de livraison

Une copie rejouable, le journal, les reçus utiles, les quatre CSV avec manifeste et Atlas.pbix actualisé. Montrez 14 faits, 35 articles et 115,500 TND, puis les deux zones historiques.

Ce que je dois pouvoir dire

« Voici le défaut observé, la règle, ma petite modification, le contrôle qui aurait révélé l'erreur, puis la deuxième revue. Voici une vente du 7 et une du 8 janvier. Voici le chiffre dans Power BI et dans le contrôle local. »

ARRIVÉE / DES COMPÉTENCES OBSERVABLES

Ce que j'ai construit et ce que je peux refaire

Je sais…	Je le prouve en…
Guider un agent	Donnant objectif, fichiers permis, règle et contrôle.
Préparer des ventes	Expliquant le sort d'une ligne et son unité.
Lire une source SQL	Montrant le SELECT, son paramètre et son résultat réel.
Organiser un entrepôt	Retrouvant une vente et ses trois descriptions.
Garder une histoire	Choisissant la version à la date et rejouant sans doubler.
Construire un rapport	Reliant quatre tables et comparant SQL à Power BI.
Réviser une correction	Conservant revue, essai et deuxième lecture.

Une confiance explicable

La prochaine fois qu'une demande paraît trop grande, commencez par un petit contrat et une preuve. Vous n'avez pas appris toutes les possibilités de Python, de SQL ou de Power BI ; vous avez appris un chemin que vous pouvez raconter, vérifier et reprendre.

Pour aller plus loin, après le socle : ajouter un autre graphique avec un titre et une unité corrects ; écrire un cas d'essai nouveau ; expliquer le chemin d'une vente à une personne qui n'a pas suivi le cours. Toute généralisation de l'historique rétroactif demande une conception supplémentaire.